AI Model Training
for Intrusion Detection Systems

A Complete Guide from Data Collection to Deployment

With a Practical Example from SALAMANDA WIDS

SALAMANDA Network Intrusion Detection System

Version 2.0

Table of Contents

1. Introduction to AI in Intrusion Detection

An Intrusion Detection System (IDS) monitors network traffic for suspicious activity. Traditional IDS tools use static rules (signatures) to match known attacks. While effective against known threats, they fail against novel or zero-day attacks.

Machine Learning (ML) adds a layer of intelligence. Instead of hardcoded rules, an ML model learns patterns from data — it can identify anomalies that no human has written a rule for.

Why ML for Network Security?

- Detects unknown/novel attack patterns (zero-day detection)
- Adapts to changing network behavior over time
- Reduces false positives through confidence scoring
- Classifies attack types (DoS, Probe, Brute Force, etc.)
- Works alongside traditional signature-based detection

Types of ML in IDS

Supervised Learning: Train on labeled data (normal vs. attack). This is what SALAMANDA uses.

Unsupervised Learning: Detect anomalies without labels (clustering, autoencoders).

Reinforcement Learning: Adapt detection policies based on feedback loops.

2. The Machine Learning Pipeline

Training an AI model follows a structured pipeline. Each step builds on the previous one:

Step 1: Problem Definition
What are we detecting? What classes/labels do we need?

Step 2: Data Collection
Gather labeled network traffic (normal + attacks)

Step 3: Feature Engineering
Extract numerical features from raw packets

Step 4: Model Selection
Choose an algorithm (Random Forest, Neural Network, etc.)

Step 5: Training
Feed data into the model, let it learn patterns

Step 6: Evaluation
Test on unseen data, measure accuracy/precision/recall

Step 7: Export
Convert to a deployment format (ONNX, TensorFlow Lite)

Step 8: Deployment
Run inference on live traffic in real-time

> *Key Insight: The model is only as good as its training data. Garbage in = garbage out. Data quality matters more than algorithm choice.*

3. Step 1 — Problem Definition

Before writing any code, define clearly what the model should detect:

For SALAMANDA WIDS:

- Input: Network traffic features (packet rates, protocols, byte counts)
- Output: Classification into Normal, DoS, Probe, R2L, or U2R
- Goal: "e95% accuracy with <5% false positive rate
- Constraint: Must run in <10ms per prediction (real-time requirement)

Attack Categories (NSL-KDD Standard):

• Normal (Class 0): Legitimate network traffic

• DoS (Class 1): Denial of Service — SYN floods, ICMP floods, UDP floods

• Probe (Class 2): Network scanning — port scans, ARP scans, ping sweeps

• R2L (Class 3): Remote to Local — brute force login, unauthorized access

• U2R (Class 4): User to Root — privilege escalation, buffer overflow

4. Step 2 — Data Collection & Preparation

You need labeled examples of both normal and attack traffic. Sources:

Public Datasets:

- NSL-KDD — Improved version of KDD Cup 99 (41 features, 5 classes)
- CICIDS2017 — Modern attacks (DDoS, brute force, web attacks)
- UNSW-NB15 — 49 features, 9 attack types
- CSE-CIC-IDS2018 — Updated with newer attack vectors

Synthetic Generation (SALAMANDA Approach):

When real attack data is unavailable or insufficient, generate synthetic samples based on known statistical distributions of attack patterns:

```
# Generate DoS attack samples
X_dos, y_dos = generate_samples(
    n=1500, label=1,       # 1500 samples, class 1 (DoS)
    duration=(0, 2),        # Very short connections
    src_bytes=(0, 100),     # Minimal payload
    count=(200, 512),       # Many connections in 2s
    serror_rate=(0.8, 1.0)  # 80-100% SYN errors
)
```

Data Splitting:

Always split data before training to get honest evaluation:

- Training Set (80%): Model learns from this
- Test Set (20%): Evaluate final performance — NEVER train on this

5. Step 3 — Feature Engineering

Raw packets are bytes. ML models need numbers. Feature engineering transforms packets into meaningful numerical vectors.

SALAMANDA Model v1 — WiFi Features (5 features):

Extracted from 802.11 wireless frames:

- packet_rate: Packets per second from a device
- deauth_ratio: Fraction of deauth frames (high = DoS)
- beacon_ratio: Fraction of beacon frames (high = AP spoofing)
- unique_channels: How many channels a device probes (high = scanning)
- avg_signal: Normalized signal strength

SALAMANDA Model v2 — Network Features (10 features):

Extracted from TCP/IP layer (inspired by NSL-KDD):

- duration: How long the connection lasted
- protocol_type: TCP (0), UDP (1), or ICMP (2)
- src_bytes: Bytes sent from source
- dst_bytes: Bytes received from destination
- land: Same src/dst host and port (1=yes)
- wrong_fragment: Corrupted fragments (anomaly indicator)
- urgent: Urgent flag count
- count: Connections to same host in 2-second window
- srv_count: Connections to same service in 2-second window
- serror_rate: Percentage of SYN errors (high = SYN flood)

Feature Normalization:

StandardScaler transforms features to mean=0, std=1. This ensures packet_rate (range 0-100) doesn't dominate serror_rate (range 0-1):

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_normalized = scaler.fit_transform(X_train)
```

6. Step 4 — Model Selection

Different algorithms have different strengths. For IDS, we prioritize speed and interpretability:

Random Forest (SALAMANDA's Primary Model)

- Ensemble of 100-150 decision trees that vote on classification
- Handles non-linear boundaries well
- Fast inference (<1ms per prediction)
- Resistant to overfitting with enough trees
- Provides feature importance ranking

Gaussian Naive Bayes (SALAMANDA's Fallback)

- Assumes features are independent (naive assumption)
- Extremely fast — good for resource-constrained environments
- Less accurate but reliable baseline

Other Options (Not used in SALAMANDA):

- Neural Networks: Higher accuracy on large datasets, slower inference
- SVM: Good for binary classification, struggles with >3 classes
- XGBoost/LightGBM: Gradient boosting — often wins competitions
- Autoencoders: Unsupervised anomaly detection

> *SALAMANDA uses Random Forest because it offers the best tradeoff between accuracy (>97%), speed (<1ms), and explainability for a real-time IDS.*

7. Step 5 — Training the Model

Training is where the model learns patterns from data. For a Random Forest:

What Happens During Training:

1. The algorithm creates N decision trees (e.g., 150)
2. Each tree sees a random subset of the training data (bagging)
3. Each tree split considers a random subset of features
4. Trees learn to separate classes by finding optimal split thresholds
5. Final prediction = majority vote across all trees

Training Code:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", RandomForestClassifier(
        n_estimators=150,    # 150 trees
        max_depth=10,        # Max tree depth
        min_samples_leaf=3,  # Min samples per leaf
        class_weight="balanced",  # Handle imbalanced classes
        random_state=42,
        n_jobs=-1  # Use all CPU cores
    ))
])

pipeline.fit(X_train, y_train)  # This is the training step
```

Hyperparameters Explained:

- n_estimators: More trees = better accuracy but slower (150 is a sweet spot)
- max_depth: Deeper trees = more complex patterns but risk overfitting
- min_samples_leaf: Prevents trees from memorizing individual examples
- class_weight='balanced': Adjusts for imbalanced classes (fewer attack samples)

8. Step 6 — Evaluation & Metrics

After training, evaluate on the TEST set (data the model has never seen):

Key Metrics:

- Accuracy: % of correct predictions overall
- Precision: Of all 'attack' predictions, how many were actually attacks?
- Recall: Of all real attacks, how many did we catch?
- F1-Score: Harmonic mean of precision and recall
- False Positive Rate: Normal traffic incorrectly flagged as attacks

Example Results (SALAMANDA v2 Random Forest):

```
=== Classification Report ===
              precision  recall  f1-score  support
Normal           0.98     0.99     0.98      600
DoS              0.99     0.99     0.99      300
Probe            0.97     0.96     0.97      200
R2L              0.95     0.94     0.95      160
U2R              0.93     0.91     0.92      140

Macro F1: 0.9620
Accuracy: 0.9743
```

Confusion Matrix:

Shows what the model predicted vs. reality. Diagonal = correct predictions:

```
Predicted !’  Normal  DoS  Probe  R2L  U2R
Normal          594    2     3     1    0
DoS               1  297     1     1    0
Probe             3    1   192     3    1
R2L               2    0     4   150    4
U2R               4    0     2     6  128
```

> *For IDS, Recall (catching real attacks) is more important than Precision. A missed attack is worse than a false alarm.*

9. Step 7 — Export & Deployment

Models trained in Python need to run in production (Node.js, C++, mobile). ONNX is the universal format:

What is ONNX?

Open Neural Network Exchange — an open format that lets you train in Python (scikit-learn, PyTorch) and run inference anywhere (JavaScript, C#, Java, mobile).

Export Code:

```
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType

# Define input shape: batch_size x num_features
initial_type = [("float_input", FloatTensorType([None, 10]))]

# Convert trained pipeline to ONNX
onnx_model = convert_sklearn(
    pipeline,
    initial_types=initial_type,
    target_opset=17
)

# Save to file
with open("models/wids_rf_v2.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())
```

Why ONNX?

- Language-agnostic: Run in JavaScript, Python, C++, Java, C#
- Hardware-optimized: Leverages CPU SIMD, GPU, NPU acceleration
- Small file size: SALAMANDA's model is ~200KB
- Fast inference: Sub-millisecond predictions

10. Step 8 — Runtime Inference

In production, SALAMANDA runs the ONNX model on every network packet:

Inference Pipeline:

Packet Arrives !’ Extract Features !’ Normalize !’ ONNX Model !’ Class + Confidence !’ Alert

JavaScript Inference (Node.js with onnxruntime-node):

```
import * as ort from "onnxruntime-node";

// Load model once at startup
const session = await ort.InferenceSession.create("models/wids_rf_v2.onnx");

// For each packet:
async function classify(features) {
  const input = new Float32Array(features);  // 10 features
  const tensor = new ort.Tensor("float32", input, [1, 10]);
  const feeds = { [session.inputNames[0]]: tensor };
  const results = await session.run(feeds);

  const classIndex = Number(results[session.outputNames[0]].data[0]);
  const probs = results[session.outputNames[1]].data;
  const confidence = probs[classIndex];

  return { classIndex, confidence };
  // classIndex: 0=Normal, 1=DoS, 2=Probe, 3=R2L, 4=U2R
}
```

Confidence Thresholding:

SALAMANDA only triggers alerts when confidence "e 92%. This dramatically reduces false positives:

```
const { classIndex, confidence } = await classify(features);
if (classIndex > 0 && confidence >= 0.92) {
  generateAlert(classIndex, confidence);
}
```

11. Complete Example: SALAMANDA WIDS

Here's the full training process end-to-end as implemented in SALAMANDA:

Step 1: Generate Training Data

```
import numpy as np

# Normal traffic: low rate, minimal errors
X_normal = generate(3000, duration=(0,300), count=(1,20),
                    serror_rate=(0, 0.05))

# DoS attack: short bursts, many connections, high SYN errors
X_dos = generate(1500, duration=(0,2), count=(200,512),
                 serror_rate=(0.8, 1.0))

# Combine all classes
X = np.vstack([X_normal, X_dos, X_probe, X_r2l, X_u2r])
y = np.concatenate([y_normal, y_dos, y_probe, y_r2l, y_u2r])
```

Step 2: Split & Train

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y)

pipeline.fit(X_train, y_train)
```

Step 3: Evaluate

```
y_pred = pipeline.predict(X_test)
print(classification_report(y_test, y_pred))
# Accuracy: 97.4%, Macro F1: 0.962
```

Step 4: Export & Deploy

```
onnx_model = convert_sklearn(pipeline, initial_types=initial_type)
# Save !’ models/wids_rf_v2.onnx (200KB)
# Load in Node.js !’ classify live traffic in <1ms
```

13. Improving the Model

Use Real Data:

Replace synthetic data with captured traffic from SALAMANDA's packet capture. Label normal vs. attack periods to create a dataset specific to your environment.

Add More Features:

- Payload entropy (encrypted vs. plaintext)
- Time-of-day patterns
- DNS query frequency and domain length
- TLS certificate anomalies

Ensemble Methods:

Combine multiple models — if both Random Forest AND Naive Bayes agree it's an attack, confidence is higher. SALAMANDA already does this with its 3-model architecture.

Online Learning:

Update the model periodically with new labeled data from dismissed alerts (false positives become negative training examples).

Deep Learning:

For larger deployments, consider LSTM or Transformer models that can learn temporal patterns across packet sequences rather than individual packets.

14. Summary & Key Takeaways

1. AI training for IDS follows: Data !’ Features !’ Train !’ Evaluate !’ Deploy
2. Random Forest is ideal for IDS: fast, accurate, interpretable
3. Feature engineering matters more than algorithm choice
4. Always evaluate on unseen test data — never train and test on the same data
5. ONNX format allows training in Python, deployment anywhere
6. Confidence thresholding ("e92%) dramatically reduces false positives
7. Synthetic data is a valid starting point; real data improves accuracy
8. Multiple models (ensemble) increase reliability
9. The model must run in real-time (<10ms) for a production IDS
10. Continuous improvement: feed real-world results back into training

To retrain SALAMANDA's models:

```
cd ml/
pip install numpy scikit-learn skl2onnx
python train_nslkdd.py
```